

Tiny Digital Logic Simulator

Gowtham S, EE24B017

October 2025

Introduction

The goal of this assignment is to implement a Tiny Digital Logic Simulator in Python that reads a netlist-like text file describing logic circuits, validates its contents, determines a valid order of gate execution, and simulates the circuit's response to a set of input stimuli. The program should finally produce a JSON file suitable for rendering timing waveforms using the WaveDrom viewer.

Functions used :

2.1 `parse_netlist(text)`

This function reads the circuit description text file and splits it into sections such as INPUTS, OUTPUTS, GATES, and STIMULUS. It then reads these sections, validates syntax and semantics, and converts them into structured Python data suitable for downstream processing. This ensures that the netlist file is well-structured before proceeding to simulation. Also, here the signal names are allowed to start with numbers, underscores and alphabets, because, no explicit restriction was posed in readme.md.

2.2 `eval_gate(gate, inputs)`

This function implements basic combinational logic for the supported gate types: AND, OR, XOR, and NOT. It assumes boolean input values (True/False or 1/0) and returns the corresponding boolean output.

2.3 `topological_sort(parsed_data)`

This function ensures that all gates are ordered such that their input signals are already declared before the gate is evaluated. I used **Kahn's algorithm** for sorting. This algorithm is designed to order nodes in a directed acyclic graph (DAG) so that every node appears after all its dependencies. In the context of the circuit simulator, each gate represents a node, and the input signals of a gate define directed edges from other nodes or inputs to this gate. The algorithm starts by initializing a queue with all input signals, which have no dependencies. It then repeatedly removes a signal from the front of the queue and considers all gates that depend on it, decrementing their dependency count (indegree). Once a gate's indegree becomes zero, meaning all its inputs are now known, it is added to the evaluation order and its output signal is appended to the queue. This process continues until all gates have been ordered or a circular dependency is detected. The advantage of Kahn's algorithm is that it efficiently ensures a valid evaluation order while also detecting cycles in the dependency graph. This method was adapted here using simple Python lists and dictionaries, avoiding specialized data structures like deque or defaultdict.

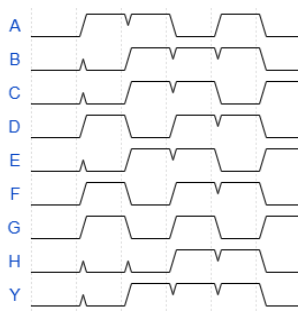
2.4 `simulate(sorted_data)`

The function iterates through each timestep defined in the STIMULUS section, assigning input signal values and computing the outputs of each gate in topological order. It produces a complete waveform table showing the values of all signals (including intermediate ones) at each time step.

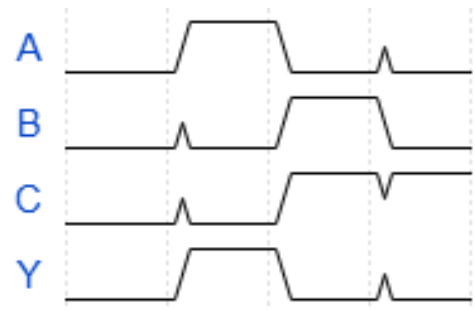
2.5 `to_wavedrom_json(data, waves)`

This function builds waveforms for each signal (inputs and outputs) over all timesteps and formats them as a JSON dictionary suitable for WaveDrom visualization.

WaveDrom timing diagrams:

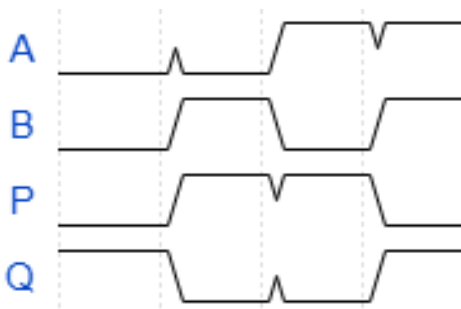


(a) For complex4.net

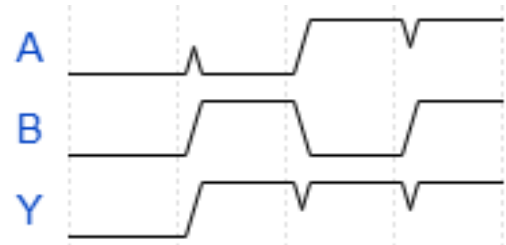


(b) For or_and_not.net

Figure 1: The outputs of the code for two predefined test cases



(a) For comp.net



(b) For simple.net

Figure 2: The outputs of the code for two custom test cases

References and Resources

While developing the topological sorting logic, I referred to:

- The GeeksforGeeks article on “Kahn’s Algorithm” to understand the principle of dependency resolution in directed acyclic graphs.
- Python’s official documentation for the argparse and json modules.

Guidelines to run the code

To run a file `simulation.net` located in the same directory and name the output as `out.json` (optional), use `python digitalsim.py simulation.net -o out.json`

Two additional test cases were created to verify the correctness of the simulator:

- `extra_test_cases/simple.net` – a small circuit testing OR gate
- `extra_test_cases/comp.net` – a more complex circuit combining all 4 gates and having resolvable dependencies

These can be run as:

```
python digitalsim.py extra_test_cases/simple.net
python digitalsim.py extra_test_cases/comp.net
```